Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 6

Team No.: 18

Project Title: Linux System Information and Resource Monitoring Tool

Team Members

Roll Number	Student Name	Signature
2520030075	THYARLA ABHINAV	
2520030524	J. ADVAITH	
2520030534	PAPAISETTY SHASHE PREETHAM	

1. Abstract

Modern Linux systems run dozens of concurrent processes competing for CPU, memory, and I/O resources, yet built-in tools such as top or htop offer limited insight into how these statistics are actually derived from kernel data structures. This project proposes the design and implementation of a Linux System Information and Resource Monitoring Tool, a user-space C application that reads and interprets live kernel-exposed data from the /proc virtual file system to report CPU utilization, memory consumption, per-process status, and disk/I/O activity in real time. The tool directly exercises core Operating Systems and Systems Programming concepts: low-level file I/O system calls (open, read, close) are used instead of buffered library calls to parse /proc/stat, /proc/meminfo, and /proc/[pid]/stat; process abstraction concepts are demonstrated by extracting PID, PPID, and process state from the process table; a multithreaded POSIX architecture, synchronized using mutexes, is used to poll CPU, memory, and disk statistics concurrently without race conditions on the shared display buffer; and signal handling (SIGINT/SIGTERM) is implemented for graceful shutdown and resource cleanup. The proposed solution will present a continuously refreshing terminal dashboard, with an optional logging mode that writes periodic snapshots to disk. By combining file abstractions, process control, and concurrency into a single cohesive utility, the project is expected to produce a functional, demonstrable system monitoring tool while reinforcing practical, hands-on understanding of how the Linux kernel exposes system state to user space.

2. Problem Statement

Standard Linux users and administrators often rely on pre-built utilities (top, htop, ps) to observe system resource usage, but these tools abstract away the underlying mechanism by which such information is obtained from the kernel. There is a need for a lightweight, transparent, user-space monitoring tool that directly demonstrates how process, memory, and CPU statistics are retrieved via the /proc file system and Linux system calls, while remaining efficient and responsive under concurrent polling of multiple resource types. This project addresses that gap by building a monitoring utility from first principles using POSIX system programming interfaces, making the OS-level mechanics fully visible and modifiable.

3. Objectives

1. To design a C-based Linux utility that retrieves real-time CPU, memory, and process information using low-level file I/O system calls on the /proc file system.

2. To implement a multithreaded polling architecture using POSIX threads, with mutex-based synchronization to safely aggregate and display statistics collected concurrently.

3. To implement signal handling (SIGINT, SIGTERM) for safe termination and proper cleanup of open file descriptors and allocated resources.

4. To provide an optional file-logging mode that periodically records system snapshots to disk for later analysis.

4. Proposed Methodology

The tool will be implemented in C on a Ubuntu Linux environment. A main controller thread will initialize a fixed-size pool of worker threads, each responsible for polling a specific resource category (CPU usage from /proc/stat, memory usage from /proc/meminfo, and per-process information from /proc/[pid]/stat and /proc/[pid]/status). Each worker will use open(), read(), and close() directly rather than buffered stdio calls, parsing the raw text data into structured statistics. A shared data structure, protected by a pthread_mutex_t, will store the latest readings; a display thread will periodically lock the mutex, read a consistent snapshot, and render it to the terminal at a fixed refresh interval. Process listings will be sorted by CPU or memory usage on request. A sigaction()-based handler will intercept SIGINT/SIGTERM to stop all worker threads cleanly, close open file descriptors, and optionally flush a final log entry before exit. Valgrind will be used during development to verify the absence of memory leaks and race conditions will be checked using stress runs with varying thread counts.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
/proc virtual file system (/proc/stat, /proc/meminfo, /proc/[pid]/stat)	Source of live kernel-exported CPU, memory, and process statistics
open(), read(), close() system calls	Low-level, unbuffered file I/O to parse /proc entries directly
POSIX Threads (pthread_create, pthread_join)	Concurrent polling of CPU, memory, and process data
pthread_mutex_t (mutex lock/unlock)	Synchronizing access to the shared statistics buffer across threads
sigaction(), SIGINT, SIGTERM	Graceful shutdown and resource cleanup on user interrupt
Process abstraction (PID, PPID, process state)	Listing and classifying active processes from the process table

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030075	THYARLA ABHINAV	CPU & memory statistics module (/proc/stat, /proc/meminfo parsing), file I/O implementation
2520030524	J. ADVAITH	Multithreaded polling architecture, mutex synchronization, signal handling, and overall integration
2520030534	PAPAISETTY SHASHE PREETHAM	Process listing module (/proc/[pid] parsing), terminal display/logging, testing & documentation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and demonstration environment
C	Core implementation language
GCC / Make	Compilation and build automation
GDB, strace, Valgrind	Debugging, syscall tracing, and memory-leak analysis
Git / GitHub	Version control and source code collaboration

8. Expected Outcome

The completed project will deliver a working command-line Linux resource monitoring tool that continuously displays live CPU utilization, memory usage, and a sorted process list in a refreshing terminal view, comparable in spirit to top but built entirely from first principles using direct system calls. The tool will handle concurrent data collection safely across multiple threads, respond correctly to interrupt signals with a clean shutdown, and optionally produce a timestamped log file of resource snapshots. The project will demonstrate practical command of process abstraction, file I/O, concurrency, and signal handling as required by the course outcomes, along with a short performance analysis comparing polling overhead at different refresh intervals and thread counts.

9. GitHub Repository

https://github.com/P-Shashe-Preetham/2520030534-2520030524-2520030075-OSSP

GitHub Repository Link: Repository Created: ☒Yes ☐No

Faculty Name and Signature
Faculty Name: Dr. K. Anusha

Signature: _____

Remarks: _____

Faculty Use
Project Approved: ☐Yes ☐No

Date: _____

Faculty Signature: _____